Week 1 — Chatbots

Overview

2 min

What's ahead

Today we'll use three different chatbots, each suited to a different task:

- **Claude** — build an interactive 3D model.
- **ChatGPT** — compare prompts, write a short story, and build a webpage and a to-do app.
- **Gemini** — build and refine a browser game.

What you'll learn

- **Prompting** — go from vague to specific, and watch wording and pushback change the answer.
- **Exploring alternatives** — branch a conversation, resubmit a prompt, or switch chatbots to compare results.
- **Building step by step** — plan first, then add one feature at a time and test as you go.
- **Staying critical** — decide what to keep, and check a claim against a real source.

Click Next to start with Activity 1.

Activity 1: Build an interactive 3D engine model

Up to 15 min (runs while we continue)

What you'll make

An interactive 3D model that explains how a car engine works.

Goals

- Turn a simple prompt into an interactive model.
- Start a build and continue with the workshop while it runs.
- Try the result and share a discovery or an unfinished attempt.

Get ready

Open Claude.ai in another tab and log in with a free or paid account.

Try it

1. Start the build. Start a new chat and send:

```
Build an interactive 3D model to explain how
a car engine works. Keep it simple.
```

"Keep it simple" helps limit the scope of the build. Keep that phrase in your first attempt, especially if you have limited usage available.

2. Continue with the workshop. Leave the Claude tab open. The build may take up to **15 minutes**. Once it starts, return to the presentation, or click Next to continue on to **Activity 2**. We'll come back to the model later.

3. Revisit and improve. When we come back to the models, open yours, try the controls, and notice what helps explain the engine. Share a discovery or something unfinished. Optional: ask for one small improvement, such as clearer labels or slower animation.

Activity 2: Stress-test a chatbot's recommendation

30 min

What you'll make

A side-by-side comparison of two conversations about choosing a personal AI agent, plus one fact you've checked for yourself.

Goals

- Notice how wording and pushback change a chatbot's recommendation.
- See how personal requirements shape which option it suggests.
- Check one factual claim against an official source.

Get ready

Log in to ChatGPT with a free or paid account. Send each prompt separately and read the response before continuing.

Try it

1. Ask, then challenge. Start a new chat and send these prompts one at a time:

```
What are AI agents?

List personal AI agents that are open-source.

Which is the best?

Are you sure?
```

2. Try different pushback. Edit that last message to try each version:

```
Really? Reconsider.

Review this critically.

Consider this carefully. I might get fired if
I'm wrong.
```

The last one is optional — adding personal pressure doesn't reliably make an answer more accurate. Did ChatGPT maintain, qualify, or change its recommendation? A changed answer does not establish that it is correct. Look at the reasons and evidence, and avoid making important decisions based on one chat.

3. Be specific. Start a new chat. Send each prompt separately:

```
Briefly explain what AI agents are in plain
language, using bullet points.

List personal AI agents that are open-source.

Which is the best? I want to try one that is
mature, safe,
and easy to install and test.

Review this critically.
```

If the list mixes ready-to-use assistants with developer toolkits, notice the difference. An open-source toolkit can still depend on paid or proprietary services underneath.

4. Compare, then check a source. Did the two conversations behave the same? Which response better addresses your needs, and why? Pick one factual claim from a recommendation — a feature, a requirement, a price — and find the project's official documentation to confirm whether it holds up.

5. Customize. Try adapting “Briefly explain this in plain language in bullets” — a phrase I use often — to a question of your own.

Activity 3: Write a short story

20 min

What you'll make

Two short story drafts of about 200 words each, from different plot ideas branched off the same brainstorm.

Goals

- Practice **Plan** → **Produce**.
- Visualize a plot with an **ASCII flowchart**.
- Branch a conversation into a new chat to develop a second idea in parallel.
- Decide for yourself which draft you prefer, and why.

Get ready

Log in to ChatGPT with a free or paid account. Keep the same chat open throughout this activity.

Try it

1. Brainstorm before drafting. Start a new chat and customize this request:

```
Help me plan a short story about two teenage
boys who get lost on
Halloween and wake up the next morning in the
school cafeteria.
Suggest three possible plots, a few sentences
each.
```

2. Choose an idea. Pick the one you like and develop it, using its number in place of 2:

```
Develop idea 2 into a brief outline.
```

3. Visualize the outline. Turn it into a simple chart:

```
Show this outline as a simple ASCII
flowchart.
```

Does the sequence make sense? Ask for a change if something is missing or confusing.

4. Ask for a short draft. Once the outline works:

```
Write a draft of about 200 words using our
outline.
Keep the language funny for teens.
```

5. Branch to try a different idea. Scroll back to the outline you made in step 2. At the end of that response, click the ••• (three dots), then choose **Branch in new chat**. This opens a new chat that continues from that point — your original chat, draft and all, stays untouched. In the branch, pick a different idea from the original three:

```
Develop idea 3 into a brief outline.
```

6. Visualize the new outline. Turn it into a chart the same way:

```
Show this outline as a simple ASCII
flowchart.
```

7. Ask for a short draft. Once the outline works:

```
Write a draft of about 200 words using this
outline.
Keep the language funny for teens.
```

Switch back to your original chat and compare the two drafts: which idea do you prefer, and why?

8. Make it yours. Pick your favorite draft and rewrite one detail yourself — a character name, a line of dialogue, or the ending — without asking the chatbot.

9. Explore further (optional). Ask the chatbot which of the two drafts it thinks works better, and why. Do you agree?

Activity 4: Create your first webpage

10 min

What you'll make

A webpage with a heading, a link, your choice of colors, and a button.

Goals

- Build in small steps: **HTML** → **a link** → **CSS** → **JavaScript**.
- Save, open, and test your webpage.
- Customize a working version with your own ideas.

Get ready

Log in to ChatGPT with a free or paid account.

Try it

1. Start small. Open a new chat and send:

```
Act as a computer science professor. I am
learning HTML. Give me the Hello World
example.
```

Glance at the code first. It looks intimidating, but it's just brackets and plain words organizing information — `<h1>` marks a heading, `<p>` marks a paragraph. You don't need to understand all of it to use it.

Click the ▶ **Preview** icon in the top-right corner of the code block to see it render.

2. Add a link. Continue in the same chat:

```
Add a short paragraph about the history of
HTML with a hyperlink.
```

Preview again. Does the link open the expected website?

3. Change its appearance. Ask:

```
Help me understand CSS by changing the font
and background color.
```

Preview to see the new look.

4. Add behavior. Ask:

```
Show me how JavaScript works by adding a
button to this page, which shows a message
when clicked.
```

Preview and click the button. Does it show the message?

5. Save and open your page. Once you're happy with it, open the Preview and click **Download Code** (top-right of the preview pane) to save it as an `.html` file, then double-click the file to open it in your browser.

No preview button on your chatbot? Create a folder called `ai-workshop` somewhere you can find again, copy the latest generated code into a plain-text editor (in TextEdit, choose

Format → Make Plain Text first), and save it as `hello.html` in that folder — use the `.html` extension rather than `.txt` (in Notepad, choose **All Files** under **Save as type**). Double-click the saved file to open it in your browser, and repeat after each change to check your work.

6. Make it yours. Customize the colors, font, or button message. Preview (or save and refresh your browser) to see the changes.

7. Explore further (optional). Ask the chatbot to point out which parts of your file are HTML, CSS, and JavaScript. Then ask how to change just the button message yourself and try that edit.

Activity 5: Build a to-do app

10 min

What you'll make

A to-do list with checkboxes, a delete icon, and a celebration when you complete a task.

Goals

- Compare a general prompt with a specific one.
- Improve and test the app in small steps.
- Plan with AI before adding a feature.
- Format prompts with Markdown.

Get ready

Log in to ChatGPT with a free or paid account.

Try it

1. Start small. Open a new chat and send:

```
Write code for a to-do list app that runs in
a web browser. Put everything in one file.
```

Click the ▶ **Preview** icon in the top-right corner of the code block to see it render. Check it to see if it works. Compare with other students to see if your apps are the same.

2. Be more specific. Revise the first prompt rather than sending a follow-up. If editing isn't available, start a new chat:

```
Write code for a to-do list that runs in a
web browser. Tasks can be entered into a box.
Once I hit Enter, the task appears below the
box, along with a checkbox. Put everything in
one file.
```

Preview it. How did the result change?

3. Improve the app. Notice this prompt uses Markdown — the `-` lines make a bulleted list, which helps the chatbot read each request as a separate item:

```
Improve this app:
- Change the title to “Zack’s todo list” (use
your own name).
- Once a task is checked, change its
background to green.
- Add an icon so that I can delete a task.
Give me the entire code.
```

Preview it.

4. Plan a feature.

```
Once I complete a task, give me some positive
feedback, such as sound and confetti. Plan
first.
```

5. Implement the plan. Read the plan, then send:

```
Implement. Give me the entire code.
```

Preview it.

6. Save and open your app. Open the Preview and click **Download Code** (top-right of the preview pane) to save it as a `todo.html` file, then double-click the file to open it in your browser.

7. Check the result. Try it in the browser, not just the preview:

- Can you add a task by pressing Enter?
- Does checking it turn the background green and trigger a celebration?
- Does the delete icon remove the right task?

8. Make it yours. Try adding a 15-minute timer or the local time in London. Plan first, then implement and test. Once you're happy with it, click **Download Code** in the Preview again to save the updated file, then open it in your browser.

Activity 6: Make a browser game

10 min

What you'll make

A shooting game with mouse controls, a score counter, and a restart option.

Goals

- Resubmit a prompt to compare different results.
- Revise a general prompt to be more specific.
- Improve and test the game one feature at a time.
- Try a different chatbot for new features (and free tokens).
- Manage context by starting a new chat and uploading your file.

Get ready

Log in to Gemini with a free or paid account.

Try it

1. Start small. Open a new chat and send:

```
Write code for a simple shooting game that
runs in the browser. Put everything in one
file.
```

Click the ⬇ **Download** icon in the top-right corner of the code pane, save it as `shooter1.html`, and open it in your browser.

2. Resubmit the prompt. Send the exact same prompt again in a new chat. Download the result as `shooter1b.html`, open it, and compare with your first version and a neighbor's.

3. Be more specific. Revise the first prompt rather than sending a follow-up. If editing isn't available, start a new chat. If you're at home, try using the microphone to dictate this prompt instead of typing it:

```
Write code for a shooting game that runs in
the browser. Put everything in one file. This
game will feature a player at the bottom of a
400x400 playing area. The player can be moved
left or right using the arrow keys. When the
space key is pressed, the player shoots a
projectile. Targets appear randomly at the
top of the screen and move down. Targets
disappear after being hit by projectiles.
```

Download it as `shooter2.html` and open it in your browser. Try moving and shooting. Keeping each version under its own filename means you can always go back to one that worked if a later change breaks something — the basic idea behind version control.

4. Change the background. Continue in the same chat:

```
Change background color to dark grey.
```

Download it as `shooter3.html` and open it in your browser.

5. Use the mouse.

```
Instead of using the keyboard, the player
moves horizontally using the mouse and shoots
when I left-click.
```

Download it as `shooter4.html` and open it in your browser.

6. Keep score.

```
Add a score counter.
```

Download it as `shooter5.html` and open it in your browser. If your game already added a score counter earlier on its own, just ask the chatbot to make it more prominent or change how points are awarded.

7. Add game over and restart.

```
If the player is hit by the targets, the game
is over. Users can restart.
```

Download it as `shooter6.html` and open it in your browser.

8. Brainstorm improvements. Start a new chat and upload the `shooter6.html` file. Then send:

```
Brainstorm ideas to improve this game.
```

As your code gets longer, a chat's history fills up with it — starting fresh with just the file keeps the conversation focused. This is called managing context.

9. Pick one idea. Choose one of the brainstormed ideas — your pick — and ask the chatbot to implement it:

```
Implement [the idea you picked]. Give me the
entire code.
```

Download it as `shooter7.html` and open it in your browser. Try it out.

10. Check the result.

- Does the mouse move the player and fire when you click?
- Does hitting a target remove it and increase the score?
- Can you restart after game over?
- Does the idea you picked work as expected?

11. Make it yours. Change the theme or target speed. Download it as `shooter8.html`, then keep numbering new versions.

If you get stuck, start a fresh chat with your current code and one requested change.

12. Explore the Gemini interface (optional). Look around the sidebar for a few other things Gemini can do:

- **Scheduled tasks** — set up a prompt to run automatically on a repeating schedule, like a daily summary.
- **Customize** — personalize how Gemini responds by saving facts about you or your preferences.
- **Videos** — generate short video clips from a text description.
- **Notebooks** — upload your own documents and ask Gemini questions grounded only in those sources.

Assignments

5 min

Prepare for next session

- Create a new Gmail account.
- Install the Claude add-in for Word and Excel.
- Install the Claude Desktop app.
- Create an account at GitHub.com.
- Install VS Code and sign in with your GitHub account.

Try and share

Try something with AI and use the Share button below to tell the group what happened. Share a discovery, a question, or an unfinished attempt. Read someone else's post and learn from each other.